

Universidad de las Américas Puebla (UDLAP)

Licenciatura en Ciencia de Datos

Actividad 13: Monitores en Java

Andrea Hernandez Huerta

ID: 175523

Heriberto Espino Montelongo

ID: 175199

Profesor: Dr. Victor Giovanni Morales Murillo

Curso: Programación Concurrente

Fecha de entrega: 25 de octubre de 2026

Índice general

1. Cola compartida con lista enlazada **1**

1.1. Entradas 1

1.2. Procesamiento 1

1.3. Salidas y verificación 1

2. Formas de utilizar `synchronized` **3**

2.1. Entradas y procesamiento 3

2.2. Salidas 3

3. Interbloqueo entre monitores **5**

3.1. Entradas y procesamiento 5

3.2. Prueba de escritorio 5

3.3. Resultado 5

4. Solución del interbloqueo **7**

4.1. Modificación 7

4.2. Prueba de escritorio 7

4.3. Comparación 8

5. Métodos adicionales de `Thread` **9**

5.1. Entradas y procesamiento 9

6. Pruebas y conclusión **11**

6.1. Verificación 11

6.2. Conclusión 11

Índice de figuras

1.1. Código y salida del programa de cola enlazada. 2

2.1. Código y salida de las tres formas de sincronización. 4

3.1. Código y evidencia real del interbloqueo. 6

4.1. Código y salida real del intercambio sin interbloqueo. 8

5.1. Código y salida de los métodos adicionales de Thread. 10

Índice de cuadros

2.1. Objetos y monitores utilizados en la venta de boletos. 3

3.1. Secuencia que produce interbloqueo. 5

4.1. Comparación entre los dos programas de intercambio. 8

5.1. Función de cada método adicional. 9

6.1. Comprobaciones automáticas de las cinco ejecuciones por programa. 11

1. Cola compartida con lista enlazada

Instrucción. Implementar una cola enlazada compartida con `Node`, `head`, `tail`, `enqueue()` y `dequeue()` sincronizados. Dos productores generan elementos identificados por su nombre y dos consumidores los retiran sin duplicaciones ni pérdidas.

1.1 Entradas

Se utilizan dos productores y dos consumidores. Para la prueba, cada productor inserta diez elementos (`P1-1` a `P1-10`, y `P2-1` a `P2-10`). Los consumidores retiran diez elementos cada uno, para un total de veinte.

1.2 Procesamiento

El monitor es el objeto `Cola`. `enqueue()` crea un nodo, actualiza `head` y `tail` y llama a `notifyAll()`. En `dequeue()`, el ciclo `while (head == null)` ejecuta `wait()` si la cola está vacía. Cuando hay un elemento, se retira el primero y se actualizan los enlaces. `synchronized` impide dos modificaciones simultáneas de la lista.

1.3 Salidas y verificación

Cada inserción y retirada muestra su identificador. Al finalizar, el programa comprueba que estén presentes los veinte identificadores distintos y muestra `Sin perdidas ni duplicados: true`. La comprobación no depende del orden en el que el sistema programe los hilos.

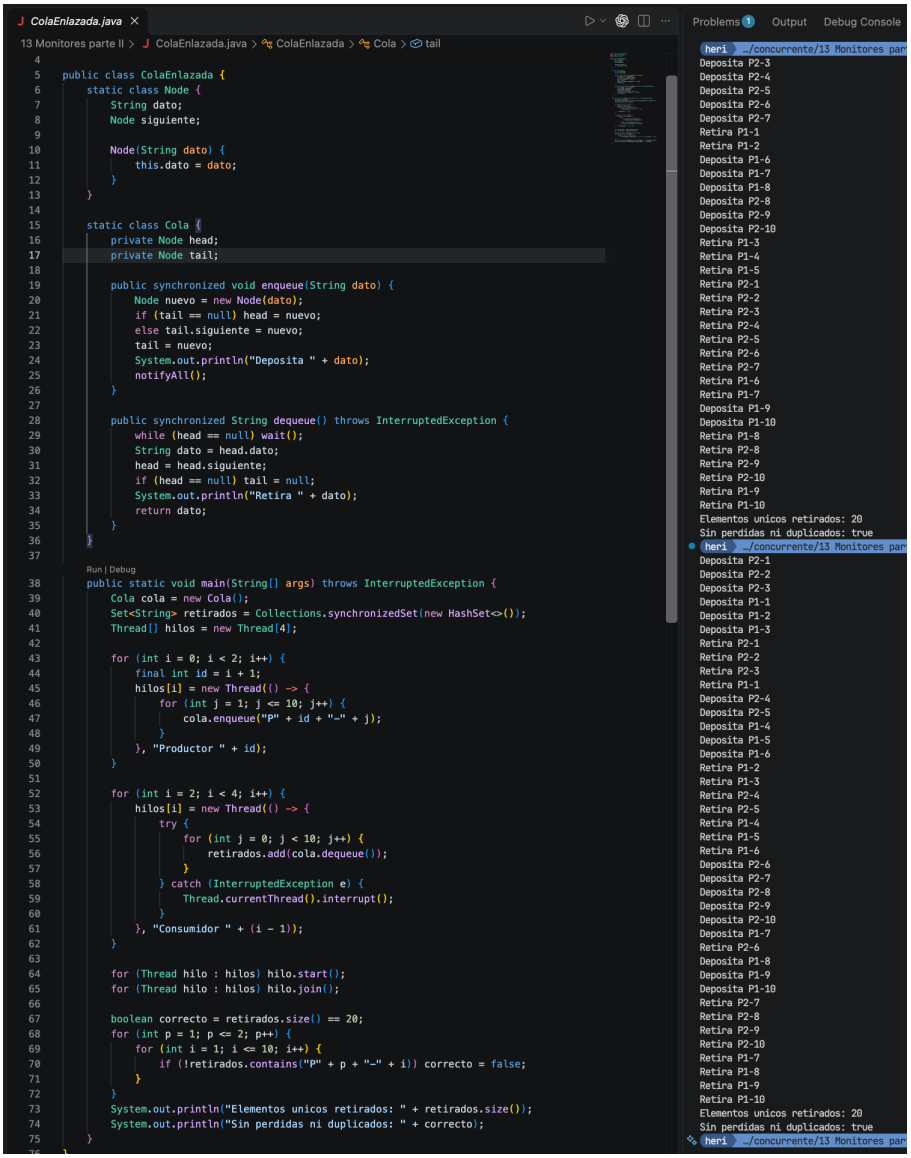


Figura 1.1: Código y salida del programa de cola enlazada.

2. Formas de utilizar synchronized

Instrucción. Comparar un método de instancia sincronizado, un bloque `synchronized(this)` y un método `static synchronized` para reservar boletos de dos eventos.

2.1 Entradas y procesamiento

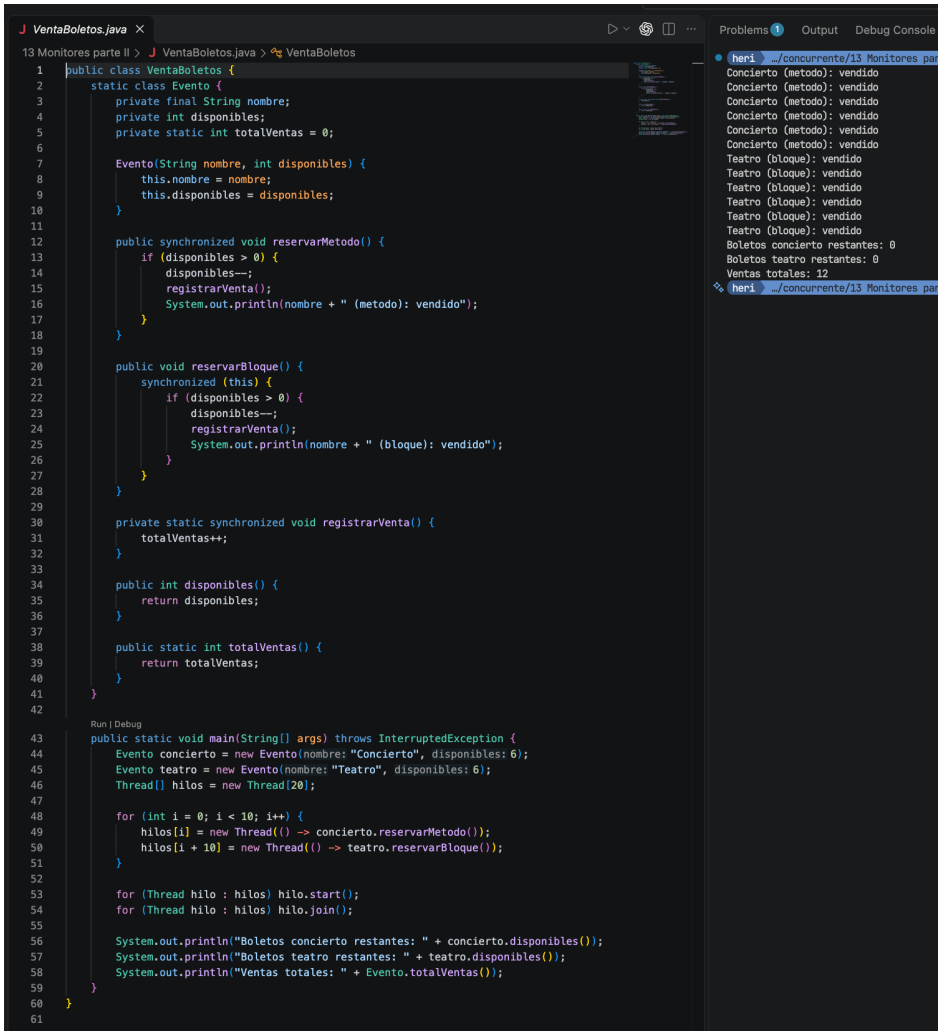
Se simulan un concierto y un teatro con seis boletos disponibles cada uno. Diez hilos intentan reservar boletos para cada evento, por lo que como máximo se venden doce boletos.

Cuadro 2.1: Objetos y monitores utilizados en la venta de boletos.

Forma de sincronización	Dato y cerrojo protegidos
Método de instancia `synchronized`	El monitor del objeto `concierto` protege sus boletos disponibles durante todo el método.
Bloque `synchronized(this)`	El monitor del objeto `teatro` protege solamente las instrucciones del bloque de reserva.
Método `static synchronized`	El monitor `Evento.class` protege el contador de ventas compartido entre eventos.

2.2 Salidas

La cantidad de boletos nunca se vuelve negativa y la suma de ventas corresponde a ambos eventos. En esta prueba se esperan cero boletos restantes por evento y doce ventas totales.



```java
public class VentaBoletos {
    static class Evento {
        private final String nombre;
        private int disponibles;
        private static int totalVentas = 0;

        Evento(String nombre, int disponibles) {
            this.nombre = nombre;
            this.disponibles = disponibles;
        }

        public synchronized void reservarMetodo() {
            if (disponibles > 0) {
                disponibles--;
                registrarVenta();
                System.out.println(nombre + " (metodo): vendido");
            }
        }

        public void reservarBloque() {
            synchronized (this) {
                if (disponibles > 0) {
                    disponibles--;
                    registrarVenta();
                    System.out.println(nombre + " (bloque): vendido");
                }
            }
        }

        private static synchronized void registrarVenta() {
            totalVentas++;
        }

        public int disponibles() {
            return disponibles;
        }

        public static int totalVentas() {
            return totalVentas;
        }
    }

    Run | Debug
    public static void main(String[] args) throws InterruptedException {
        Evento concierto = new Evento(nombre: "Concierto", disponibles: 6);
        Evento teatro = new Evento(nombre: "Teatro", disponibles: 6);
        Thread[] hilos = new Thread[20];

        for (int i = 0; i < 10; i++) {
            hilos[i] = new Thread(() -> concierto.reservarMetodo());
            hilos[i + 10] = new Thread(() -> teatro.reservarBloque());
        }

        for (Thread hilo : hilos) hilo.start();
        for (Thread hilo : hilos) hilo.join();

        System.out.println("Boletos concierto restantes: " + concierto.disponibles());
        System.out.println("Boletos teatro restantes: " + teatro.disponibles());
        System.out.println("Ventas totales: " + Evento.totalVentas());
    }
}
```

```
Concierto (metodo): vendido
Concierto (metodo): vendido
Concierto (metodo): vendido
Concierto (metodo): vendido
Concierto (metodo): vendido
Concierto (metodo): vendido
Teatro (bloque): vendido
Teatro (bloque): vendido
Teatro (bloque): vendido
Teatro (bloque): vendido
Teatro (bloque): vendido
Teatro (bloque): vendido
Boletos concierto restantes: 0
Boletos teatro restantes: 0
Ventas totales: 12
```

Figura 2.1: Código y salida de las tres formas de sincronización.

3. Interbloqueo entre monitores

Instrucción. Crear dos objetos `Cell`, `p` y `q`, y ejecutar simultáneamente `p.swap(q)` y `q.swap(p)`. Introducir una pausa al obtener el primer monitor y explicar el interbloqueo.

3.1 Entradas y procesamiento

Los valores iniciales son `p = 10` y `q = 20`. Cada `swap()` es sincronizado, así que el hilo adquiere primero el monitor del objeto receptor. Luego intenta obtener el monitor del otro objeto. Se añade una pausa y una barrera de dos hilos para mostrar de manera reproducible la espera circular.

3.2 Prueba de escritorio

Cuadro 3.1: Secuencia que produce interbloqueo.

Paso	Hilo	Operación	Monitor que posee	Monitor que espera
1	A	`p.swap(q)`	`p`	Ninguno
2	B	`q.swap(p)`	`q`	Ninguno
3	A	Intenta entrar en `q`	`p`	`q`
4	B	Intenta entrar en `p`	`q`	`p`

3.3 Resultado

Ningún hilo puede adquirir su segundo monitor, porque el otro lo conserva. El programa usa hilos *daemon* y espera un tiempo limitado para detectar

la situación sin dejar abierta la demostración; **esto no resuelve el interbloqueo**.

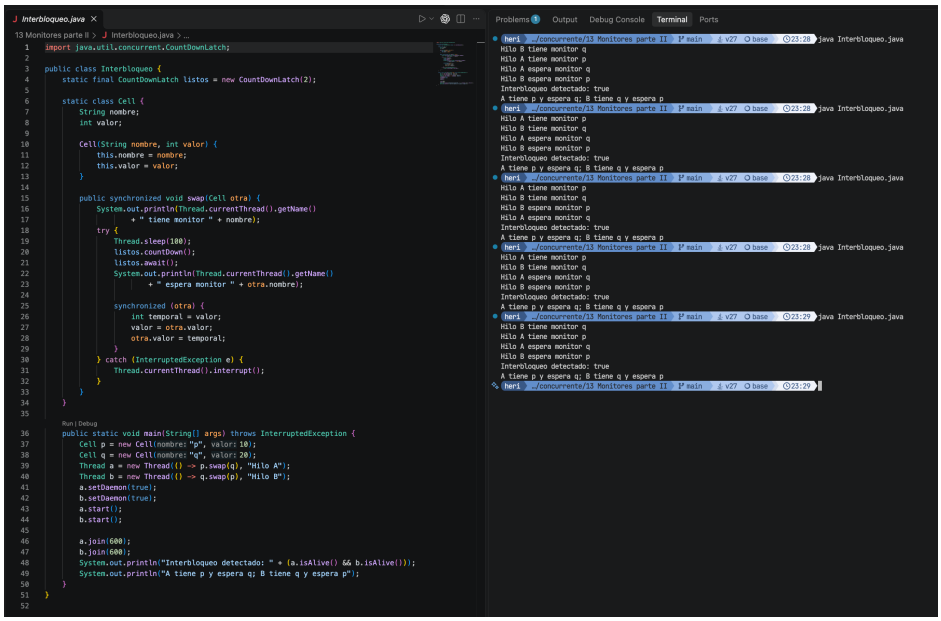


Figura 3.1: Código y evidencia real del interbloqueo.

4. Solución del interbloqueo

Instrucción. Corregir la adquisición de monitores usando `System.identityHashCode()` y ejecutar de nuevo los dos intercambios.

4.1 Modificación

En lugar de adquirir primero el monitor propio, los dos hilos calculan el código de identidad de `p` y `q` y toman ambos monitores **en el mismo orden**: primero el de código menor y después el de código mayor. Una vez obtenidos, el objeto de código menor ejecuta `doSwap()`.

También se contempla el caso poco frecuente de códigos iguales utilizando un monitor adicional de desempate. Así, tampoco se forma una espera circular en ese caso.

4.2 Prueba de escritorio

Si el código de `p` es menor, tanto A como B solicitan primero `p`, luego `q`. A puede conseguir ambos monitores mientras B espera `p`. A los libera al terminar; entonces B obtiene ambos. Se realizan dos intercambios y los valores regresan a `p = 10` y `q = 20`.

4.3 Comparación

Cuadro 4.1: Comparación entre los dos programas de intercambio.

Aspecto	Sin orden	Con orden fijo
Primer monitor	Cada hilo toma el suyo	Ambos usan el de código menor
Espera circular	Posible y demostrada	Evitada por el mismo orden
Fin de los dos hilos	Pueden quedar bloqueados	Terminan normalmente

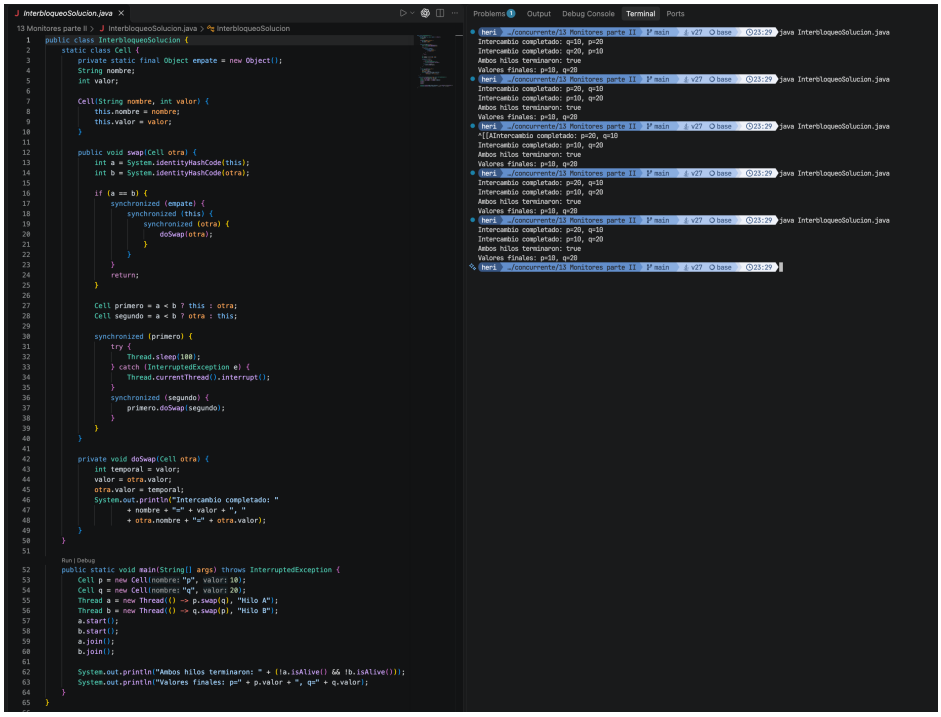


Figura 4.1: Código y salida real del intercambio sin interbloqueo.

5. Métodos adicionales de Thread

Instrucción. Mostrar `interrupt()`, `yield()` y `holdsLock()` mediante ejemplos cortos.

5.1 Entradas y procesamiento

Cuadro 5.1: Función de cada método adicional.

Método	Descripción y observación
`interrupt()`	Interrumpe un hilo que está en `sleep()`; aparece el mensaje de interrupción al capturar `InterruptedException`.
`Thread.yield()`	Sugiere ceder el procesador. No obliga al planificador ni garantiza un orden específico.
`Thread.holdsLock(objeto)`	Devuelve `false` fuera de `synchronized(objeto)` y `true` dentro, en el hilo actual.

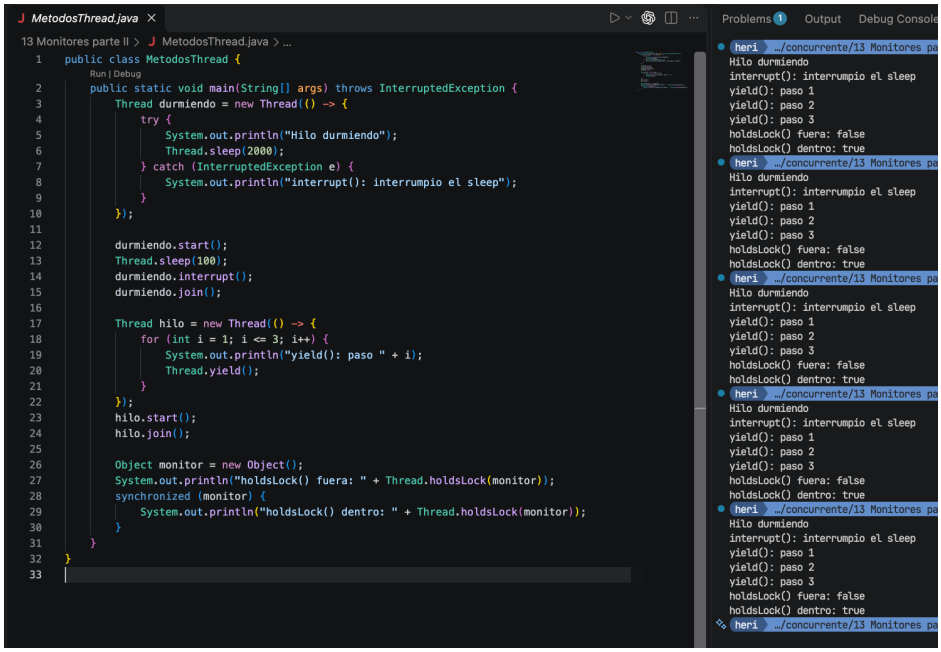


Figura 5.1: Código y salida de los métodos adicionales de Thread.

6. Pruebas y conclusión

6.1 Verificación

Cada programa se ejecuta cinco veces mediante el flujo de pruebas del repositorio. Se comprueban las propiedades concretas indicadas a continuación; la secuencia de mensajes entre hilos puede cambiar de una ejecución a otra.

Cuadro 6.1: Comprobaciones automáticas de las cinco ejecuciones por programa.

Programa	Condición comprobada
Cola enlazada	Los veinte elementos aparecen exactamente una vez al retirarlos.
Venta de boletos	Ningún evento vende más de seis entradas; se realizan doce ventas.
Interbloqueo	Los dos hilos permanecen bloqueados en la demostración.
Solución ordenada	Ambos hilos terminan y los dos intercambios restauran los valores originales.
Métodos de `Thread`	Se observa la interrupción, la ejecución con `yield()` y los dos valores de `holdsLock()`.

6.2 Conclusión

Los monitores proporcionan exclusión mutua sobre objetos compartidos, mientras que `wait()` y `notifyAll()` coordinan la espera cuando una condición todavía no se cumple. Los métodos de instancia sincronizados y los bloques `synchronized(this)` adquieren el monitor del objeto; un método `static synchronized` utiliza el de la clase.

Un monitor por sí solo no evita todos los interbloqueos: adquirir varios monitores en distinto orden puede crear una espera circular. Establecer el mis-

mo orden para todos los hilos evita esa situación. Finalmente, `interrupt()`, `yield()` y `holdsLock()` permiten observar otras operaciones propias de los hilos en Java.